Manager-worker Mandelbrot

Manager-worker parallelism

- In this unit we see an implementation of the Mandelbrot program using a Manager-worker pattern:
  - One MPI process is a “manager” process which hands out work to worker processes
  - All other processes are “workers” which request work from the manager process
  - Workers request more work when they have finished their current task

mandelbrot_mpi_mw.c

```
/*************************************************************
Description: Program to calculate the Mandelbrot set using
             a manager-worker pattern

Notes: compile with: mpicc -o mandelbrot_mpi_mw mandelbrot_mpi_mw.c -lm
       run with:     sbatch run_mandelbrot_mw.sh


*************************************************************/

#include <stdio.h>
#include <complex.h>
#include <stdbool.h>
#include <math.h>
#include <mpi.h>

/* Number of intervals on real and imaginary axes*/
#define N_RE 3000
#define N_IM 2000

/* Number of iterations at each z value */
int nIter[N_RE+1][N_IM+1];

/* Points on real and imaginary axes*/
float z_Re[N_RE+1], z_Im[N_IM+1];

/* Domain size */
const float z_Re_min = -2.0; /* Minimum real value*/
const float z_Re_max =  1.0; /* Maximum real value */
const float z_Im_min = -1.0; /* Minimum imaginary value */
const float z_Im_max =  1.0; /* Maximum imaginary value */

/* Set to true to write out results*/
const bool doIO = true;
/*****************************************************************************/
```

Number of intervals on each axis

Number iterations at each z value

Points along real and imaginary axes

Domain size

Switch I/O on and off

mandelbrot_mpi_mw.c

```
/*****************************************************************************/

int main(int argc, char *argv[]){

  /* MPI related variables */
  int myRank;        /* Rank of this MPI process */
  int nProcs;        /* Total number of MPI processes*/
  int nextProc;      /* Next process to send work to */
  MPI_Status status; /* Status from MPI calls */
  int endFlag=-9999; /* Flag to indicate completion*/

  /* Timing variables */
  double start_time, end_time;

  /* Loop indices */
  int i,j;

  MPI_Init(&argc, &argv);

  /* Record start time */
  start_time=MPI_Wtime();

  /* Get job size and rank information */
  MPI_Comm_rank(MPI_COMM_WORLD, &myRank);
  MPI_Comm_size(MPI_COMM_WORLD, &nProcs);

  if (myRank==0){
    printf("Calculating Mandelbrot set with %d processes\n", nProcs);
  }

  /* Initialise to nIter zero as we will use a reduce to collate results into this array*/
  for (i=0; i<N_RE+1; i++){
    for (j=0; j<N_IM+1; j++){
      nIter[i][j]=0;
    }
  }
```

Initialise MPI

Record the start time
(seconds since arbitrary point)

Get size of and rank in
`MPI_COMM_WORLD`

Report number of
processes used

mandelbrot_mpi_mw.c

```
if (myRank==0){
  printf("Calculating Mandelbrot set with %d processes\n", nProcs);
}

/* Initialise to nIter zero as we will use a reduce to collate results into this array*/
for (i=0; i<N_RE+1; i++){
  for (j=0; j<N_IM+1; j++){
    nIter[i][j]=0;
  }
}

/* Points on real axis */
for (i=0; i<N_RE+1; i++){
  z_Re[i] = ( ( (float) i)/ ( (float) N_RE)) * (z_Re_max - z_Re_min) + z_Re_min;
}

/* Points on imaginary axis */
for (j=0; j<N_IM+1; j++){
  z_Im[j] = ( ( (float) j)/ ( (float) N_IM)) * (z_Im_max - z_Im_min) + z_Im_min;
}
```

Initialise `nIter` to zero

Populate `z_Re` with points on real axis

Populate `z_Im` with points on imaginary axis

mandelbrot_mpi_mw.c

```
/* Points on imaginary axis */
for (j=0; j<N_IM+1; j++){
  z_Im[j] = ( ( (float) j)/ ( (float) N_IM)) * (z_Im_max - z_Im_min) + z_Im_min;
}

// Manager process
if ( myRank == 0 ){
  // Hand out work to worker processes
  for (i=0; i<N_RE+1; i++){
    // Receive request for work
    MPI_Recv(&nextProc, 1, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
    // Send i value to requesting process
    MPI_Send(&i,        1, MPI_INT, nextProc,       100,         MPI_COMM_WORLD);
  }
  // Tell all the worker processes to finish (once for each worker process = nProcs-1)
  for (i=0; i<nProcs-1; i++){
    // Receive request for work
    MPI_Recv(&nextProc, 1, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
    // Send endFlag to finish
    MPI_Send(&endFlag,     1, MPI_INT, nextProc,       100,         MPI_COMM_WORLD);
  }
}

// Worker Processes
else {

  while(true){

    // Send request for work
    MPI_Send(&myRank, 1, MPI_INT, 0, 100+myRank, MPI_COMM_WORLD);
    // Receive i value to work on
    MPI_Recv(&i,      1, MPI_INT, 0, 100       , MPI_COMM_WORLD, &status);

    if (i==endFlag){
      break;
    } else {
      calc_vals(i);
    }

  } // while(true)
} // else worker process
```

2) Manager process receives request from next available worker

3) Manager sends next `i` value to worker

1) Worker process sends request to manager process

4) Worker receives next `i` value from manager

mandelbrot_mpi_mw.c

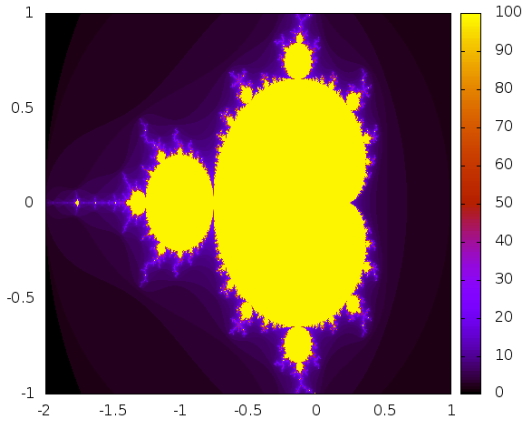
```
/* Points on imaginary axis */
for (j=0; j<N_IM+1; j++){
  z_Im[j] = ( ( (float) j)/ ( (float) N_IM)) * (z_Im_max - z_Im_min) + z_Im_min;
}


// Manager process
if ( myRank == 0 ){
  // Hand out work to worker processes
  for (i=0; i<N_RE+1; i++){
    // Receive request for work
    MPI_Recv(&nextProc, 1, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
    // Send i value to requesting process
    MPI_Send(&i,        1, MPI_INT, nextProc,       100,         MPI_COMM_WORLD);
  }
  // Tell all the worker processes to finish (once for each worker process = nProcs-1)
  for (i=0; i<nProcs-1; i++){
    // Receive request for work
    MPI_Recv(&nextProc, 1, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
    // Send endFlag to finish
    MPI_Send(&endFlag,     1, MPI_INT, nextProc,       100,         MPI_COMM_WORLD);
  }
}

// Worker Processes
else {
  while(true){

    // Send request for work
    MPI_Send(&myRank, 1, MPI_INT, 0, 100+myRank, MPI_COMM_WORLD);
    // Receive i value to work on
    MPI_Recv(&i,      1, MPI_INT, 0, 100       , MPI_COMM_WORLD, &status);

    if (i==endFlag){
      break;
    } else {
      calc_vals(i);
    }

  } // while(true)
} // else worker process
```

When all iterations have been handed out send flag to end

Loop over `i` is replaced by a while loop

mandelbrot_mpi_mw.c

Calculate number of iterations for all `j` values at a given `i` value




```
void calc_vals(int i){

  /* Maximum number of iterations*/
  const int maxIter=100;

  /* Value of Z at current iteration*/
  float complex z;
  /*Value of z at iteration zero*/
  float complex z0;

  int j,k;

  /* Loop over imaginary axis */
  for (j=0; j<N_IM+1; j++){
    z0 = z_Re[i] + z_Im[j]*I;
    z  = z0;

    /* Iterate up to a maximum number or bail out if mod(z) > 2 */
    k=0;
    while(k<maxIter){
      nIter[i][j] = k;
      if (cabs(z) > 2.0)
        break;
      z = z*z + z0;
      k++;
    }
  }

}
```

Store results in `nIter`

mandelbrot_mpi_mw.c

```
    } // while(true)
  } // else worker process

  /* Communicate results so rank 1 has all the values */
  do_communication(myRank);

  /* Write out results */
  if (doIO && myRank==1 ){
    printf("Writing out results from process %d \n", myRank);
    write_to_file("mandelbrot.dat");
  }

  /* Record end time */
  end_time=MPI_Wtime();

  /* Report elapsed time */
  printf("Proc %d elapsed time (seconds): %f\n", myRank, end_time-start_time);

  MPI_Finalize();

  return 0;

}
```

Communicate results

Write results to file

mandelbrot_mpi_mw.c

```
void do_communication(int myRank){

  MPI_Group worldGroup, workerGroup;
  MPI_Comm  workerComm;
  int zeroArray={0};

  int sendBuffer[(N_RE+1)*(N_IM+1)];
  int receiveBuffer[(N_RE+1)*(N_IM+1)];
  int index=0;
  int i, j;

  // Get a group handle for the world group
  MPI_Comm_group(MPI_COMM_WORLD, &worldGroup);
  // Form a new group excluding world group rank 0
  MPI_Group_excl(worldGroup, 1, &zeroArray, &workerGroup);
  // Create a communicator for the new group
  MPI_Comm_create(MPI_COMM_WORLD, workerGroup, &workerComm);

  /* Pack nIter into a 1D buffer for sending*/
  for (i=0; i<N_RE+1; i++){
    for (j=0; j<N_IM+1; j++){
      sendBuffer[index]=nIter[i][j];
      index++;
    }
  }

  /* call MPI_reduce to collate all results on world group process 1
      The world group rank zero process does not make this call */
  if (myRank != 0 ){
    MPI_Reduce(&sendBuffer, &receiveBuffer, (N_RE+1)*(N_IM+1), MPI_INT, MPI_SUM, 0, workerComm);
  }

  /* Unpack receive buffer into nIter */
  index=0;
  for (i=0; i<N_RE+1; i++){
    for (j=0; j<N_IM+1; j++){
      nIter[i][j]=receiveBuffer[index];
      index++;
    }
  }

  /* Free the group and communicator */
  if (myRank != 0 ){MPI_Comm_free(&workerComm);}
  MPI_Group_free(&workerGroup);

}
```

Make a new communicator with worker processes only

Pack **`nIter`** into a 1D buffer

Call **`MPI_Reduce`** with worker processes only

Unpack receive buffer into **`nIter`**

Free the new group and communicator

Write results to file:

```
void write_to_file(char filename[]){

  int i, j;
  FILE *outfile;

  outfile=fopen(filename,"w");
  for (i=0; i<N_RE+1; i++){
    for (j=0; j<N_IM+1; j++){
      fprintf(outfile,"%f %f %d \n",z_Re[i], z_Im[j], nIter[i][j]);
    }
  }
  fclose(outfile);

}
```

Write results to file:

```
    } // while(true)
  } // else worker process

  /* Communicate results so rank 1 has all the values */
  do_communication(myRank);

  /* Write out results */
  if (doIO && myRank==1 ){
    printf("Writing out results from process %d \n", myRank);
    write_to_file("mandelbrot.dat");
  }

  /* Record end time */
  end_time=MPI_Wtime();

  /* Report elapsed time */
  printf("Proc %d elapsed time (seconds): %f\n", myRank, end_time-start_time);

  MPI_Finalize();

  return 0;

}
```

Record the end time
(seconds since arbitrary point)

Write elapsed time
from all processes

Close down MPI

Unit summary

- In this unit we have seen an MPI implementation of the Mandelbrot program which uses a manager- worker algorithm to distribute the work
- This should scale better than a domain decomposition because manager-worker can load- balance at run time (like a dynamic schedule in OpenMP)
- We will test this in this week's workshop